CAB BOOKING SYSTEM

Project Title: Cab Booking System
Technology Stack: MERN Stack (MangoDB, Express.js, React.js, Node.js)
Developed By: Golagana Harikrishna

TABLE OF CONTENTS

INTRODUCTION

Project Title: Cab Booking
Team Members: Golagana Harikrishna
Roles and Responsibilities: Developed the full-stack MERN application including user authentication, Mongoose schemas, driver dispatch flow, HTML5 canvas maps, and deployed the project on Render and MongoDB Atlas.

PROJECT OVERVIEW

The Cab Booking System is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) to simplify online cab booking. It enables users to securely register, book rides, track ride status, make simulated payments, and submit ratings. Drivers can manage ride requests, while administrators monitor the system through a dashboard. The application uses a client-server architecture with secure authentication and cloud-based MongoDB Atlas, demonstrating modern full-stack web development practices.

Purpose

The primary purpose of the **Cab Booking System** is to provide a secure, efficient, and user-friendly platform for online cab booking. It enables passengers to register, book rides, track ride status, and make simulated payments with ease. Drivers can manage ride requests and update ride progress, while administrators can monitor users, rides, and overall system performance through a dedicated dashboard. The project also demonstrates the practical implementation of full-stack web development technologies and software engineering principles to deliver a reliable and scalable transportation solution.

Objectives

The Cab Booking System has been developed with the following objectives:

- To enable passengers to register, log in, and request rides easily.
- To allow drivers to accept ride requests and update ride status.
- To simulate fare calculation and payment processing.
- To provide ride history and rating functionality.
- To implement responsive user interfaces for different devices.
- To ensure secure storage of user information using MongoDB Atlas.

Features

The Cab Booking System provides several advanced features that improve user convenience and demonstrate modern web application development techniques.

User Registration and Login: The application provides secure registration and login facilities for Riders and Drivers. Users create accounts using their personal details, and passwords are securely encrypted before storage.

Driver Registration: Drivers can register their vehicle information, maintain availability status, and receive booking requests from passengers.

Ride Booking: Passengers can enter pickup and destination locations, select booking information, and confirm their ride requests through an intuitive booking interface.

Ride Tracking: The system provides a visual representation of ride progress using animated HTML5 Canvas maps that simulate vehicle movement between locations.

Payment Simulation: The project includes a mock payment system that simulates online payment processing without integrating actual banking services.

Rating System: Passengers can rate drivers after completing rides, helping improve service quality and customer satisfaction.

Admin Dashboard: Administrators can monitor total users, registered drivers, active rides, completed bookings, and revenue generated through the system.

Scope

The **Cab Booking System** provides a complete web-based solution for online ride booking, including user registration, secure authentication, ride booking, ride management, payment simulation, ride history, and ratings. It offers separate modules for riders, drivers, and administrators to efficiently manage their respective activities. The application is deployed using **Render** and **MongoDB Atlas**, making it accessible over the internet. Designed primarily for educational purposes, the project demonstrates modern full-stack web development concepts and can be further enhanced with features such as real-time GPS tracking and live payment integration.

SYSTEM ARCHITECTURE

System Architecture Overview

The Cab Booking System follows a three-tier architecture consisting of the Presentation Layer (React.js), Application Layer (Node.js & Express.js), and Data Layer (MongoDB Atlas). This architecture separates the user interface, business logic, and database, ensuring better scalability, security, and maintainability. The frontend communicates with the backend through REST APIs, while the backend processes requests, interacts with the database, and returns the required responses. This design provides efficient, secure, and reliable application performance.

Frontend Architecture

The frontend of the Cab Booking System is developed using React.js with Vite, providing a fast and interactive user interface through reusable components. Bootstrap 5 ensures a responsive design, while React Router enables smooth navigation between pages. The application includes separate dashboards for riders, drivers, and administrators, along with pages for login, registration, ride booking, ride tracking, payment, and ratings. The frontend communicates with the backend through REST APIs to display data dynamically.

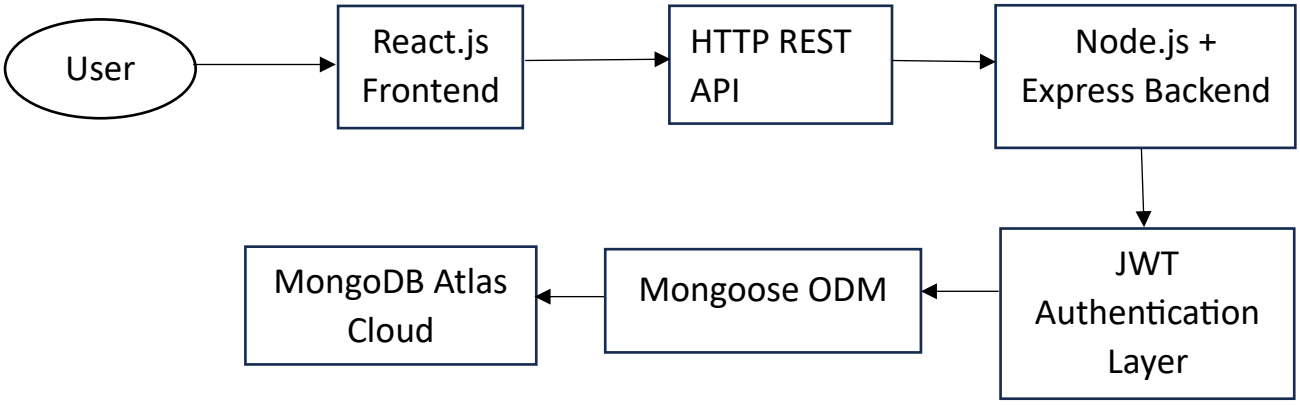
Backend Architecture

The backend of the Cab Booking System is developed using Node.js and Express.js. It handles authentication, ride management, database operations, and API requests. The backend uses controllers and middleware to process business logic, validate requests, and secure protected routes using JWT authentication. Its modular structure improves maintainability and scalability.

Database Architecture

The Cab Booking System uses MongoDB Atlas, a cloud-based NoSQL database, to store and manage application data. Mongoose ODM is used for schema definition, validation, and database operations. The system organizes data into collections such as User, Car, Ride, Payment, and Rating, which store user details, vehicle information, booking records, payment details, and customer feedback efficiently.

Architecture Diagram



SETUP INSTRUCTIONS

Prerequisites

Node.js (v18 or higher)
Git
A web browser (Google Chrome or Microsoft Edge recommended for voice features)
A MongoDB Atlas account

Installation

Step 1: Clone the Repository

Open the terminal and run:
git clone https://github.com/Harikrishna7274/Cab-Booking.git
This downloads the project to your local system.

Step 2: Open the Project

Navigate to the project directory:
cd Cab-Booking
The project consists of two main folders:
client – React.js frontend
server – Express.js backend

Step 3: Install Backend Dependencies

Navigate to the server folder and install the required packages:
cd server
npm install

Step 4: Install Frontend Dependencies

Open a new terminal, navigate to the client folder, and install the required packages:
cd client

npm install
After the installation is complete, both the frontend and backend are ready to run.

Environment Variables

Environment variables are used to store sensitive configuration details securely outside the source code. Create a .env file in the server directory and add the following:

PORT=8000
MONGO_URI=Your_MongoDB_Atlas_Connection_String
JWT_SECRET=YourSecretKey
NODE_ENV=production

Database Setup

Create a Cluster: Create a new cluster in MongoDB Atlas to store the application data.

Configure Network Access: Add 0.0.0.0/0 in the Network Access section to allow the application to connect to the database.

Create a Database User: Create a database user with Read and Write permissions.

Configure the Connection String: Copy the MongoDB Atlas connection string and paste it into the MONGO_URI variable in the .env file.

Once configured, the backend server will automatically connect to the MongoDB Atlas database when the application starts.

FOLDER STRUCTURE

Client

The **Client** folder contains the React.js frontend of the Cab Booking System and manages the user interface and user interactions.

src/components/ – Contains reusable components such as Rnav.jsx, Dnav.jsx, and Anav.jsx, which provide navigation for Riders, Drivers, and Administrators.

src/pages/ – Contains the main application pages, including Home.jsx, Rhome.jsx, Dhome.jsx, Ahome.jsx, RequestRide.jsx, and RideTracking.jsx, each responsible for a specific feature of the system.

src/index.css – Defines the global styles, responsive layout, animations, and overall appearance of the application.

Server

The **Server** folder contains the backend of the Cab Booking System and

manages API requests, business logic, authentication, and database operations.
controllers/ – Contains the business logic for users, rides, cars, and admin operations.
db/ – Connects the application to the MongoDB Atlas database.
middlewares/ – Includes JWT middleware for user authentication and route protection.
models/ – Defines Mongoose schemas for User, Car, Ride, Payment, and **Rating**.
routes/ – Maps API endpoints to their respective controller functions.
server.js – The main file that starts the Express server, configures middleware, and connects to the database.

RUNNING THE APPLICATION

After completing the installation and configuration, start both the frontend and backend servers to run the application.

Frontend (Local Development)

Navigate to the client directory and run:
npm run dev
Once the development server starts, open the application in your browser at:
http://localhost:5173

Backend (Local Development)

Navigate to the server directory and run:
npm run dev
This starts the Express.js backend server and establishes a connection with the MongoDB Atlas database.

Production Build

To generate an optimized production build of the frontend, navigate to the client directory and run:
npm run build
This creates optimized static files that are ready for deployment.

API DOCUMENTATION

The Cab Booking System uses RESTful APIs to facilitate communication between the frontend and backend. The following APIs handle user

authentication, ride management, and payment processing.

User Registration API

Endpoint: POST /api/users/register

Registers a new Rider or Driver by accepting user details such as name, email, password, and role. On successful registration, the API returns a confirmation message along with the user information.

Request: Name, Email, Password, Role

Response: Success message and user details

Status Codes: 201 Created, 400 Bad Request

User Login API

Endpoint: POST /api/users/login

Authenticates registered users using their email and password. Upon successful authentication, a JWT token and user profile are returned.

Request: Email, Password

Response: JWT token and user profile

Status Codes: 200 OK, 401 Unauthorized

Ride Request API

Endpoint: POST /api/rides/request

Allows a rider to create a new ride request by providing the required booking details.

Request: Car ID, Pickup Location, Drop Location, Booking Date

Response: Created ride details

Status Codes: 201 Created, 401 Unauthorized

Rider History API

Endpoint: GET /api/rides/rider/history

Retrieves the complete ride history of the authenticated rider.

Response: List of previous rides

Status Codes: 200 OK

Accept Ride API

Endpoint: POST /api/rides/accept

Enables a driver to accept a pending ride request and updates the ride status.

Request: Ride ID

Response: Updated ride details

Status Codes: 200 OK, 400 Bad Request

Payment API

Endpoint: POST /api/rides/pay

Processes the mock payment for a completed ride and records the payment information.

Request: Ride ID, Amount

Response: Payment confirmation

Status Codes: 200 OK

AUTHENTICATION AND SECURITY

The Cab Booking System uses **JWT (JSON Web Token)** and **bcryptjs** to provide secure user authentication and protect application data.

Login

Users log in using their email and password. The server verifies the credentials and generates a JWT token upon successful authentication, allowing access to protected resources.

Registration

New users register by providing their details through the registration form. The system checks for duplicate email addresses, encrypts the password using bcryptjs, and stores the user information securely in the database.

JWT Authentication

Protected API routes are secured using JWT authentication. The middleware validates the Bearer token sent in the request header and grants access only to authenticated users.

Password Encryption

User passwords are encrypted using bcryptjs with 10 salt rounds before being stored in the database, ensuring secure password protection.

USER INTERFACE

The Cab Booking System provides a simple, responsive, and user-friendly interface for Riders, Drivers, and Administrators.

Home Page

The Home Page serves as the landing page of the application. It includes a welcome banner, portal links for different user roles, and animated background effects using HTML5 Canvas.

Login Page

The Login Page allows registered users to access the system by entering their email address and password.

Registration Page

The Registration Page enables new users to create an account by providing their details and selecting either the Rider or Driver role.

Rider Dashboard

The Rider Dashboard displays nearby cab availability, ride statistics, recent bookings, and provides quick access to ride booking and ride history.

Request Ride Page

The Request Ride Page allows riders to enter pickup and drop-off locations, select ride options, and submit a booking request. It also displays the route using an SVG-based map.

Driver Dashboard

The Driver Dashboard displays pending ride requests, assigned rides, and allows drivers to update ride status and manage their availability.

Admin Dashboard

The Admin Dashboard provides an overview of the system, including total revenue, active trips, registered users, and driver online/offline status for effective system monitoring.

TESTING

The Cab Booking System was tested using different testing methods to ensure that all features function correctly and provide a reliable user experience.

Unit Testing

Individual controllers and database schemas were tested independently using sample data to verify their functionality and ensure accurate results.

Integration Testing

The complete application workflow was tested by performing user registration, login, ride booking, driver acceptance, payment processing, and rating submission to ensure smooth interaction between all modules.

API Testing

All REST API endpoints were tested using **Postman** to verify request handling, responses, authentication, and HTTP status codes.

Manual Testing

The application was manually tested on desktop and mobile browsers to verify

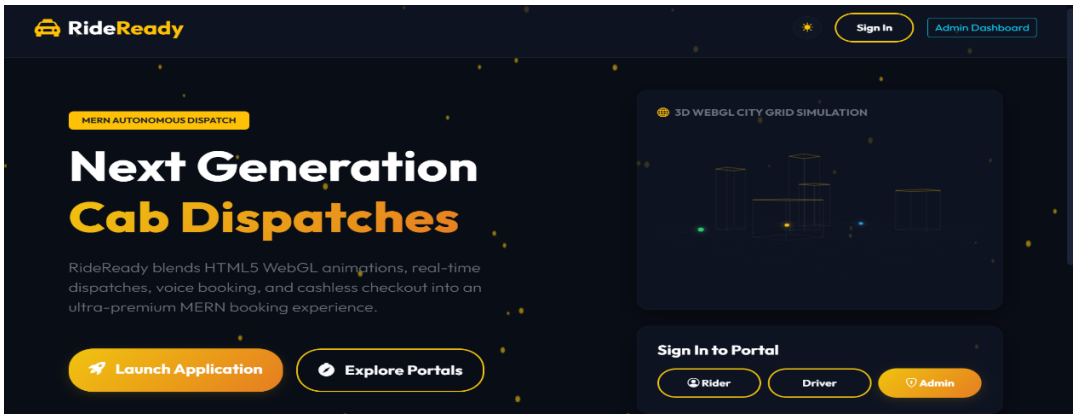
features such as user registration, login, ride booking, ride tracking, payment, and theme switching, ensuring proper functionality and responsiveness.

DEMO

This section presents the major interfaces of the Cab Booking System. The screenshots demonstrate the application's functionality and provide a visual understanding of the user interface. Replace the placeholders below with the corresponding screenshots in the final document.

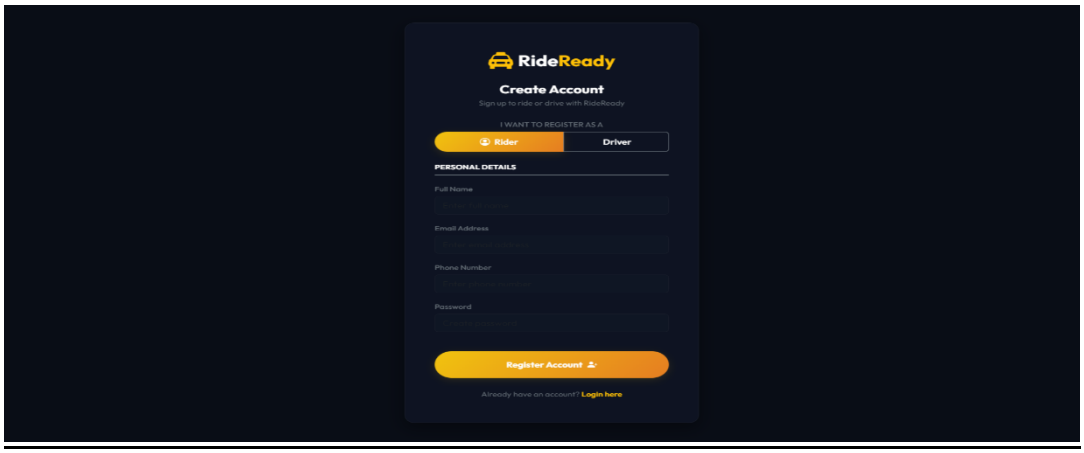
Home Page

The Home Page is the main entry point of the application. It provides navigation links for Riders, Drivers, and Administrators, along with an interactive user interface.



Login Page

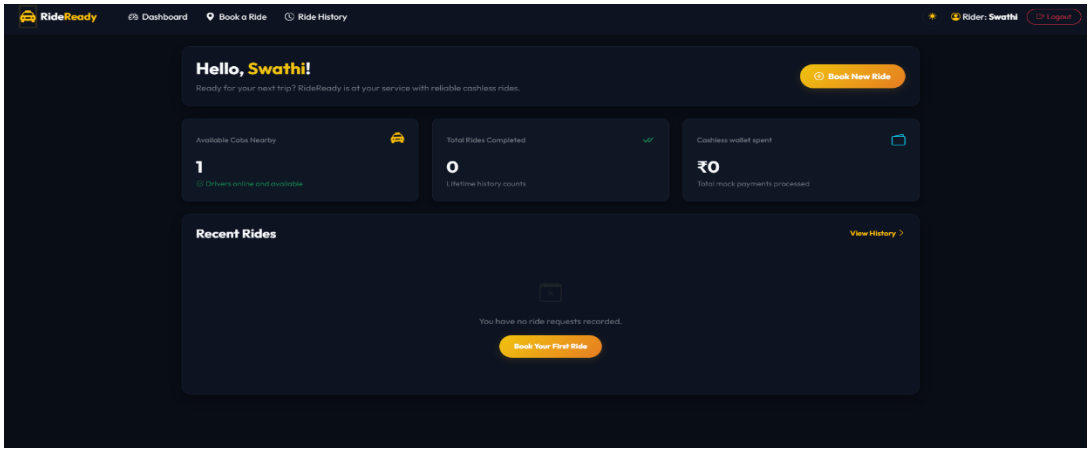
The Login Page allows registered users to securely access the system using their email address and password.



Dashboard

The Dashboard provides role-specific features and information for Riders,

Drivers, and Administrators after successful login.



Live Website

The Cab Booking System has been successfully deployed on Render and can be accessed using the following URL:

Live Website:

https://cab-booking-client.onrender.com

GitHub Repository

The complete source code of the project is available in the GitHub repository below. It includes both the frontend and backend code along with the project structure and configuration files.

GitHub Repository:

https://github.com/Harikrishna7274/Cab-Booking

KNOWN ISSUES

The current version of the Cab Booking System has a few known limitations that may affect the user experience under certain conditions.

Render Cold Start: Since the backend is hosted on Render's free tier, the server may become inactive after a period of inactivity. As a result, the first request may take 30–50 seconds to respond while the server restarts.

Browser Compatibility: The voice-based booking feature uses the Web Speech API, which is not fully supported in browsers such as Mozilla Firefox. For the best experience, Google Chrome or Microsoft Edge is recommended.

FUTURE ENHANCEMENTS

The Cab Booking System can be further enhanced by adding advanced features to improve functionality and user experience.

Real-Time Location Tracking: Integrate WebSockets to provide live synchronization of driver and rider locations during a trip.

Online Payment Gateway: Integrate secure payment gateways such as Stripe or Razorpay to support real-time online payments.

Google Maps Integration: Incorporate the Google Maps API to provide live navigation, route optimization, and accurate location tracking.

These enhancements will improve the application's performance, usability, and make it more suitable for real-world deployment**.**

CONCLUSION

The Cab Booking System is a modern web application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) to simplify the process of booking and managing cab services. The project successfully demonstrates the practical application of full-stack web development technologies, RESTful APIs, cloud database integration, secure authentication, and responsive user interface design.

The system enables riders to register, log in, book rides, track ride status, make simulated payments, and submit ratings. Drivers can manage ride requests and update ride statuses, while administrators can monitor users, rides, and overall system performance through a dedicated dashboard.

The project also incorporates modern features such as JWT-based authentication, bcrypt password encryption, responsive design, voice-assisted booking, theme switching, and an AI Travel Assistant. These features enhance usability and demonstrate current industry practices.

Although some advanced functionalities, including real-time GPS tracking and live payment integration, are not included in the current version, the modular architecture allows these features to be added in future updates.

Overall, the Cab Booking System successfully meets its objectives and serves as a strong academic project, showcasing technical knowledge, software engineering principles, and practical problem-solving skills.